

论文模板 09 · 架构演化与遗留系统改造

配套素材见 [00-项目素材库.md](#)。本主题竞争压力小、容易出彩，因为大多数考生没有真实的架构演进经历。BASE HOME 的「AWS SDK v1/v2 渐进共存」「契约驱动三语言」都是极佳素材。

一、真题常见问法

变体 A（架构演化）

请论述你参与的某系统架构的演化过程，包括演化的驱动力、演化策略以及演化过程中如何控制风险。

变体 B（遗留系统改造）

请论述你如何对遗留系统进行改造，如何在不中断业务的前提下完成架构升级。

变体 C（架构风格）

请论述你参与的某系统采用的架构风格，说明选择依据及该风格的优缺点。

变体 D（可维护性/复用）

请论述你如何通过架构设计提升系统的可维护性与可复用性。

判断：变体 A/B 用「SDK v1/v2 渐进共存 + 契约统一」；变体 C 用「分层 + 事件驱动 + 服务化」的风格组合；变体 D 用「契约单一来源 + 版本统一治理 + 模块化」。

二、摘要模板（约 300 字）

本文论述了本人参与的 H.I.S. BASE HOME 旅游业务平台的架构演化过程。

该平台由早期单体应用逐步演进为多模块、多部署单元的架构，

期间需同时支撑 Java、Go、PHP 三类调用方，

并兼容大量基于旧版 AWS SDK 的既有代码。

本人担任架构师，负责演化路径规划、技术债治理与风险控制。

针对【新旧技术栈长期并存】、【接口契约在多语言环境下漂移】、

【演化过程不能中断业务】三大挑战，

本文遵循【渐进式演进】、【契约先行】、【依赖显式化】三个原则，

通过能力边界划分实现 AWS SDK v1/v2 的有序共存，

通过 protobuf 契约单一来源统一三端接口定义，

通过统一版本管理插件把隐式依赖转为显式契约。

系统在演化期间保持连续可用，【 填效果】，验证了策略的有效性。

三、正文五段框架

第 1 段·项目背景与演化驱动力（约 400 字）

演化类论文的核心是「驱动力」——为什么必须变？

BASE HOME 的架构演化并非为了技术而技术，而是由三类真实驱动因素推动：

驱动一：调用方技术栈扩张 系统最初仅服务 Java 侧的 Web 应用，随着集团平台化推进，周边 Go 平台与既有 PHP 系统也需接入。若为每种语言各写一份接口定义，字段口径必然漂移。这迫使我们在接口层做出根本性改变。

驱动二：数据形态复杂化 旅游业务的文档、亲权同意书、消息线程等数据字段演进频繁，关系型模型的 DDL 变更成本越来越高，促使我们引入 NoSQL 与对象存储，形成混合存储架构。

驱动三：云平台能力演进 AWS SDK v2 提供了 v1 所没有的 DynamoDB 增强客户端（`DynamoDbEnhancedClient`），而既有代码大量基于 v1。全量重写风险过高，必须寻找演进路径。

这三类驱动力共同决定：**架构必须演化，但不能推倒重来。**

第 2 段·演化策略与过程（约 800 字，本文核心）

第一原则：渐进式演进，拒绝大爆炸式重写

面对 AWS SDK v1 到 v2 的迁移，我们评估了两条路径：

方案	优点	风险
一次性全量迁移到 v2	技术栈彻底统一	涉及大量既有代码，回归范围不可控；业务不能停
v1/v2 按能力边界共存 （采纳）	风险可控，新能力不受阻	双版本长期共存，依赖体积与维护成本上升

我们采纳了共存方案，并确立清晰的能力边界划分规则： - **新增能力一律走 v2**——DynamoDB 增强访问、STS AssumeRole 临时凭证 - **既有能力暂留 v1**——已稳定运行的 S3/STS 调用不做无谓改动 - **客户端构建集中收口**——在 `DynamoDbConfig`、`AwsConfig` 中统一构建，业务代码不直接创建客户端

这条规则的价值在于：它把「迁移」从一个需要排期的项目，变成了一个**随新增功能自然发生的渐进过程**。每新增一个功能，技术债就减少一点，而不需要专门立项。

这是我在本项目最重要的架构判断：技术债的偿还方式，不一定是集中偿还，也可以是"增量稀释"。

第二原则：契约先行，用单一来源约束多方

面对多语言调用方的接口漂移问题，我在 `base-home-spec` 模块中确立 `protobuf` 为**唯一契约源**，通过构建流水线（`build.sh`：`build-java` → `build-go` → `build-php` → `generate-all`）生成 Java、Go、PHP 三语言产物及 OpenAPI 文档。

演化视角的价值：契约先行不仅解决了当下的接口一致性问题，更重要的它是**可演化性的基础**——因为契约集中，接口的版本演进、废弃标记、兼容性检查都只需要在一个地方进行。分散的接口定义无法演化，只能重写。

第三原则：依赖显式化，把隐式契约变成可控状态

项目初期各模块独立声明依赖版本，部分采用动态区间（`protobufVersion = '[3,4]'`、`grpcVersion = 'latest.release'`）。这带来一个隐蔽的风险：**构建产物不可复现**——同一份代码在不同时间、不同机器上可能解析到不同版本。

我引入了独立的 `base-home-versions` Gradle 插件作为统一版本源，各模块通过 `apply plugin: 'base-home.versions'` 接入，并使用 `resolutionStrategy { force ... }` 锁定关键组件。

这不是一次预防性的重构——**风险已经真实发生：**`proto-google-common-protos` 被解析到 2.64.0-rc1 后，gRPC 服务启动直接失败。这次故障印证了「隐式版本契约」是架构层面的缺陷，而非偶发问题。

第 3 段 · 演化过程中的具体改造（约 700 字）

改造一：数据存储的演化

从单一关系库演进为混合存储：Aurora MySQL 承载强一致交易（79 张表）、DynamoDB 承载高弹性半结构化数据（5 张表）、S3 承载文件实体。

演化策略：新数据类型直接落在新存储，既有数据不做迁移。例如文档与亲权同意书这类新增业务，直接以 `document` / `parental` 表落 DynamoDB，而既有交易数据保持在 MySQL。这样避免了大规模数据迁移的风险，同时让架构逐步具备弹性能力。

代价与应对：跨存储的一致性通过事件驱动链路达成最终一致——DynamoDB Stream / SQS 触发的 14 个 Lambda 函数负责将数据同步回 MySQL。这条链路本身就是「演化」的产物：它使新旧存储可以并存而不失一致。

改造二：在线与离线处理的分离

批量作业从与在线业务共享数据库，演进为双数据源读写分离（`spring.datasource.reader` / `writer`，按包路径隔离 Mapper）。这一改造的直接动因是：批处理峰值会推高在线接口延迟。改造后读写的物理隔离使两者互不干扰。

改造三：服务能力的沉淀与复用

随着 RPC 数量增长（当前 17 个），跨业务域的重复逻辑被识别并下沉：文档、消息线程、亲权同意书、序列号、旅行信息 5 类能力沉淀到 `service/shared/`，文件能力归入 `service/filesystem/`。服务端也从「每个 RPC 一个类」重构为「单一入口 `BaseHomeService` + `execute()` 统一分发」——横切关注点（日志、异常转换、上下文校验）得以集中处理。

第 4 段 • 难点与解决 (约 700 字)

架构演化主题最佳组合：

优先级	难点	适配理由
首选	③ AWS SDK v1/v2 并存	直接对应「遗留系统改造」
次选	① 一份 proto 契约驱动三语言	直接对应「架构复用与可演化性」
三选	② protobuf 版本冲突（风险真实发生）	直接对应「演化风险控制」
加分	⑤ 混合存储的形成过程	直接对应「数据架构演化」

难点示例（③，三段式）：

问题：DynamoDB 增强客户端只在 AWS SDK v2 提供，而既有代码大量基于 v1。全量迁移涉及面广、回归风险不可控，且业务不能停机等待。

方案：我判断这里的核心矛盾不是「技术选型」，而是「**如何在不停机、不失控的前提下完成技术栈演进**」。因此我没有选择一次性重写，而是按能力边界划分：新增能力走 v2，既有能力暂留 v1，客户端构建统一收口在配置类中，业务代码不直接创建客户端。这样做的深层价值是——**收口点就是未来的迁移点**。当某天要彻底移除 v1 时，只需改动配置类，而不需要遍历所有业务代码。

效果：◆ 新功能开发不再受 SDK 版本阻塞；◆ 凭证获取统一收口后，凭证类故障下降。更重要的是，技术债从「需要专门立项偿还」变成了「随新增功能自然稀释」。

第 5 段 · 总结与不足 (约 200 字)

整个演化过程历时【◆ 时间】，期间系统保持连续可用，未发生因演进导致的业务中断。

不足之处有两点：

一是 AWS SDK v1 与 v2 的并存状态尚未终结，长期维护成本仍然存在；

二是跨语言契约的破坏性变更尚未实现自动化的兼容性检测，目前依赖人工评审。

后续计划完成 SDK 版本的最终收敛，并引入契约兼容性校验。

通过本项目，我对架构演化形成了三点认识：

- ① 演化必须由业务驱动力推动，而非技术偏好；
- ② 渐进式演进的成本看似更高，但风险可控，而大爆炸式重写的风险往往是不可承受的；
- ③ 把隐式契约显式化，是让架构「可演化」的前提。

四、本项目填充点速查

论文要素	直接可写的内容
演化主线	单体 → 多模块多部署单元；单一关系库 → 混合存储；单语言 → 三语言
技术债 1	AWS SDK v1 (S3/STS) 与 v2 (DynamoDB Enhanced) 并存
技术债 2	动态版本区间 → 统一版本源 + resolutionStrategy force
技术债 3	分散接口定义 → protobuf 契约单一来源
渐进策略	新增走新版、既有留旧版、客户端构建集中收口
数据演进	新增业务直接落 DynamoDB/S3，不做大规模数据迁移
一致性保障	DynamoDB Stream / SQS → 14 个 Lambda → MySQL (最终一致)
在线离线分离	双数据源 reader/writer + 包路径隔离 Mapper

论文要素	直接可写的内容
服务重构	单入口 <code>BaseHomeService</code> + <code>execute()</code> 分发; <code>service/shared/</code> 沉淀复用
契约演进	约 90 个 proto 分 request/response/enums/type 四层, <code>type/</code> 约 42 个可复用消息
版本治理	<code>base-home-versions</code> Gradle 插件 (<code>base-home.versions.gradle</code>)
版本对齐	<code>his-common-spec</code> 2.6.0 在 Java/Go/PHP 三侧对齐
构建产物	Java → Maven; Go → module zip; PHP → Composer; 文档 → MkDocs
环境治理	DynamoDB 表名带环境前缀, 「1 账户 1 集群」
模块依赖方向	web/service → common + spec; batch → common + mdm-spec
本地研发演进	TestContainers (MySQL) + LocalStack (<code>:4566</code>) + moto (Python)
测试支撑	<code>NoopJob</code> / <code>NoopService</code> 便于集成测试与逐步替换

💡 「NoopJob / NoopService」是个隐藏的演化论据：项目中存在专门用于测试的空实现作业与服务，这说明架构在设计时就考虑了可替换性——写「可演化性」「可测试性」时可以用作佐证。

五、可用难点素材（按适配度排序）

- 1. ③ AWS SDK v1/v2 并存 ★★★★★ —— 本主题首选
- 2. ① 一份 proto 契约驱动三语言 ★★★★★ —— 可演化性与复用
- 3. ② protobuf 版本冲突（风险已发生） ★★★★★ —— 演化风险控制
- 4. ⑤ 混合存储的形成过程 ★★★★★ —— 数据架构演化
- 5. ④ 读写分离改造 ★★ —— 在线/离线分离演化

六、踩坑提醒

坑	后果	正确做法
编造「从单体拆成微服务」的戏剧性故事	高风险	本项目是模块化多仓库结构，如实写「多模块 + 独立部署单元」
只讲技术不讲驱动力	空洞	演化必须有业务/技术驱动力支撑
只讲结果不讲过程	漏考点	必须写路径选择与权衡（为什么不大爆炸式重写）
不提遗留系统的风险	漏考点	遗留改造论文必须写「风险控制」——业务不中断怎么保证
编造容器化改造 / K8s 迁移	高风险	本项目未见 K8s 证据
忘记写演化代价	缺乏架构思维	必须写并存状态的维护成本、双版本依赖体积等代价

坑	后果	正确做法
编造数据迁移规模	高风险	本项目是「新增业务落新存储」，不是「海量数据迁移」，如实写更稳

💡 **本主题的最大优势：**「渐进式演进」这个论点本身，就是软考官方教材强调的核心理念。你有一个**真实发生过的**版本冲突故障（难点 ②）来佐证「隐式契约是架构缺陷」这一判断，这种「预判风险 → 风险发生 → 已备预案」的完整叙事，在论文中非常有说服力，绝大多数考生写不出来。

配套：[01- 微服务架构设计.md](#) · [06- 系统集成与gRPC.md](#) · [07- 架构评估与质量属性.md](#)